

Bases de Datos No Relacionales

Clasificación de países europeos según predominio solar o eólico

usando PySpark

Integrantes:

Rosa Itzel Gutierrez Barrera

Nayade Velasco Acosta

Uriel Antonio Chávez Sánchez

Julián Quiroz Ramírez

The logo of the Instituto Tecnológico y de Estudios Superiores de Occidente (ITAM) is displayed in a large, bold, dark green font. The letters are stylized, with the 'I' and 'T' being particularly prominent and blocky.

Índice

· Portada	1
· Índice (Tabla de contenido)	2
· Introducción	3
· Problemas encontrados	3
· Solución	4
· Solución (diagrama con la solución)	5
· Características de la solución	7
· Obtención y almacenamiento de datos	8
· Estructuras de las tablas de la BD	9
· Resultados de las consultas y gráficas	10
· Conclusiones	11
· Bibliografía	11

Introducción

En las últimas décadas, la penetración de energías renovables en Europa ha crecido muy rápidamente, principalmente impulsada por la energía solar fotovoltaica y por la energía eólica. Sin embargo, la disponibilidad de estos recursos puede ser variable en el tiempo y por las situaciones económicas de los países, lo que hace necesario que contemos con herramientas que nos ayuden a analizar estos grandes volúmenes de datos para entender mejor los patrones.

En este proyecto vamos a utilizar PySpark para procesar un conjunto de datos con fechas horarias, así como los factores de capacidad de energía solar y eólica para los países europeos desde 1980 hasta la actualidad, proveniente del dataset “ninja_pv_wind_profiles_singleindex”. El objetivo principal es identificar qué países presentan un perfil de generación renovable solar o eólica, así como cuáles podrían considerarse mixtos o balanceados, a partir del análisis estadístico de sus factores de capacidad promedio. Para esto, diseñaremos una solución donde emplearemos las facilidades de PySpark para la lectura, transformación y agregación de los datos, además de recurrir a bibliotecas de Python para la generación de tablas y gráficos.

Problemas encontrados

Durante el desarrollo del proyecto se identificaron varios problemas relacionados tanto con la naturaleza del dataset como con la estructura.

- a) Volumen de la información: El dataset trabaja con datos horarios de varias décadas para decenas de países y de diversas tecnologías (solar fotovoltaica, eólica, entre otras). Esto genera un volumen de datos cercano a 0.5 GB, lo que resulta difícil de manejar con herramientas tradicionales (como Excel) y que nos obliga a utilizar soluciones de Big Data como PySpark.
- b) Heterogeneidad en las tecnologías por país: Al descargar el archivo, pudimos observar que la estructura no es completamente uniforme entre los distintos países. Por ejemplo, para algunos países existe una columna “XX_pv_national”, pero no aparece la columna correspondiente al viento “XX_wind_national”, mientras que en otros países si se encuentran columnas tanto de energía solar como eólica. Esto representa un problema para comparar el predominio solar o eólico de los países, ya que idealmente se requeriría contar con datos de ambas tecnologías porque la

ausencia de dichas columnas implica que no es posible calcular de manera directa el mismo conjunto de métricas para todos los países.

- c) Datos faltantes: La falta de columnas de viento para algunos países nos plantea la duda de cómo integrar estos casos para el análisis. Por lo que tendríamos que tomar una decisión para erradicar este problema, ya sea limitar el análisis principal únicamente a países que cuentan con al menos una columna de solar y una columna de eólica; o bien, completar el esquema de columnas añadiendo columnas vacías para que el DataFrame tenga la misma estructura para todos los países.

Solución

Para solucionar los problemas identificados en la sección anterior, diseñamos una solución en PySpark que nos permitirá procesar el conjunto de datos horarios de energía solar y eólica para todos los países europeos contenidos en el DataSet. La solución se implementará de la siguiente manera:

- a) Lectura del dataset con PySpark: Primero se cargará el archivo en un DataFrame utilizando las funciones vistas en clase para cargar este tipo de archivos. De esta manera, cada columna del archivo corresponderá a un país del dataset.
- b) Unificación del esquema para todos los países: Dado que no todos los países cuentan con el mismo conjunto de tecnologías en el archivo original, se optó por completar el esquema de columnas para tener una estructura homogénea. Por eso, a partir de las columnas que ya existen, identificaremos a las tecnologías de cada país, en los casos en que un país no disponga de una de estas tecnologías, crearemos una nueva columna correspondiente donde la llenaremos de valores nulos. Por ejemplo, si un país no tiene una columna como la de “XX_wind_national”, generaremos esta columna con valores nulos para este país. Esta decisión permitirá que el DataFrame final tenga en todos los países las mismas columnas de solar y eólica, sin interpretar la ausencia de datos de manera propia.
- c) Transformación a un formato analítico para cada país y tecnología: Básicamente vamos a generalizar los datos, es decir, una vez unificado el esquema, transformaremos el DataFrame original a un formato con columnas más explícitas como:

time (fecha horaria, marca de tiempo),

country (código del país, abreviación del país),
technology (tipo de tecnología, solar o eólica),
capacity_factor (valor horario del factor de capacidad).

Esta transformación nos facilitará la agrupación y los cálculos por país y por tecnología.

- d) Cálculo de métricas agregadas: Una vez teniendo el DataFrame con las respectivas soluciones implementadas, calcularemos para cada país los factores de capacidad promedio de ambas tecnologías. A partir de estos promedios se definirá un indicador de predominio que comparará la importancia relativa de la energía solar frente a la eólica en cada país. Este indicador nos ayudará a clasificar a los países en tres categorías: solares, eólicos o mixtos.

Solución (Incluir un diagrama con la arquitectura de la solución)

Fuente de datos → Ingesta PySpark → Unificación de esquema → Transformación → Agregaciones → Tablas/Gráficas → Conclusiones

Fuente de datos

Caja: “Dataset Ninja PV & Wind Profiles”

Contiene el archivo masivo con factores de capacidad horarios de energía solar fotovoltaica y eólica para múltiples países europeos.

Formato original: CSV con una columna de tiempo (`time`) y múltiples columnas del tipo `PAIS_TECNOLOGÍA` (por ejemplo, `BA_pv_national`, `BE_pv_national`, `BE_wind_national`, `BE_wind_offshore`, etc.).

Ingesta en PySpark

Caja: “Lectura del CSV con PySpark”

Proceso: se utiliza `spark.read.csv(...)` para cargar el archivo en un DataFrame de PySpark.

Resultado: un DataFrame “ancho” donde la primera columna es `time` y el resto son combinaciones país–tecnología.

Unificación del esquema (creación de columnas vacías)

Caja: “Normalización de columnas por país y tecnología”

Se identifican los países presentes y las tecnologías de interés (por ejemplo, `pv_national` y `wind_national`).

Para cada país que no tiene alguna de estas tecnologías, se crea la columna correspondiente con valores nulos (`null`).

Objetivo: que todos los países dispongan del mismo conjunto de columnas de solar y eólica, evitando interpretar la ausencia de datos como generación cero.

Transformación a formato analítico (“ancho → largo”)

Caja: “Reestructuración del DataFrame”

El DataFrame se transforma desde el formato de muchas columnas (`PAIS_TECNOLOGÍA`) a un formato “largo” con las columnas:

- `time` (marca de tiempo),
- `country` (país),
- `technology` (solar o eólica),
- `capacity_factor` (factor de capacidad horario).

Esta estructura facilita las operaciones de agrupamiento y cálculo de estadísticas por país y tecnología.

Cálculo de métricas agregadas e indicador de predominio

Caja: “Agregaciones y clasificación de países”

Con el DataFrame en formato analítico, se calculan:

- Promedios de `capacity_factor` por país y tecnología.

- Métricas adicionales como mínimos, máximos y desviaciones estándar, según se requiera.

A partir de los promedios de solar y eólica se define un indicador de predominio para cada país, que permite clasificarlo en una de tres categorías:

- Más solar,
- Más eólico,
- Mixto o balanceado.

Exportación de resultados a Pandas y generación de gráficas

Caja: “Tablas y gráficas (Pandas + Matplotlib)”

Las tablas agregadas finales se convierten a DataFrames de Pandas.

Con Matplotlib se generan gráficos de barras, comparaciones por país y representaciones visuales de los grupos (más solares, más eólicos y mixtos).

Estas visualizaciones sirven como base para la interpretación de resultados en el documento.

Resultados y conclusiones

Caja final: “Interpretación y conclusiones”

A partir de las tablas y gráficas generadas se analizan los patrones observados:

Qué países muestran predominio solar,

Cuáles son principalmente eólicos,

Qué países se consideran mixtos.

Se sintetizan estos hallazgos en conclusiones sobre el perfil renovable de los países europeos analizados y sobre la utilidad de PySpark para el tratamiento de este tipo de datos masivos.

Características de la solución

La solución para el análisis del predominio solar o eólico por país presenta varias características que la hacen adecuada para trabajar con un conjunto de datos heterogéneo y masivo.

Primero haremos todo el flujo de lectura, limpieza, transformación y agregación de los datos que se realizará con PySpark. Esto nos permitirá manejar sin problema alguno un archivo de bastante tamaño como es este. Posteriormente tendremos el esquema unificado para todos los países mediante columnas vacías, dado que el archivo original no contiene las mismas tecnologías para todas las tecnologías, la solución va a incluir una etapa de normalización donde crearemos columnas vacías para las combinaciones país-tecnología que no están presentes en aquellos países para evitar interpretar la ausencia de columnas. Posteriormente tenemos el modelo de datos analítico y flexible, el dataframe inicial en formato “ancho” pasará a un formato más “largo, más adecuado para el análisis. Esta estructura nos facilitará los agrupamientos por país, tecnología, periodo, etc. Y nos

permitirá extender el análisis sin estar modifique y modifique el modelo. Por otro lado, crearemos un indicador general que nos ayudará a comparar directamente los factores de capacidad medios de ambas tecnologías por país, a partir de este indicador se clasificarán los países en distintas categorías como: solares, eólicos y mixtos. Después, tenemos la integración con herramientas de visualización en Python, los resultados agregados se convierten posteriormente a DataFrame de Pandas para generar tablas y gráficos con Matplotlib, para poder tener la visualización de los hallazgos de una manera clara y atractiva. Por último, un flujo de trabajo que estará diseñado de forma modular donde será posible cambiar el periodo analizado, añadir nuevos países, incorporación de tecnologías sin tener que alterar la lógica del proyecto, es decir, listo para añadir o eliminar contenido sin alterar la estructura.

Obtención y almacenamiento de los datos

Utilizamos un conjunto de datos de acceso abierto que contiene perfiles horarios de energía solar y eólica desde 1980 hasta la actualidad de 32 países europeos, tanto del norte como del sur de este continente. Estos datos provienen del archivo conocido como “ninja_pv_wind_profiles_singleindex” generado a partir de modelos de Renewables.ninja y recopilado por la iniciativa Open Power System Data.

La descarga del archivo se realizó a través de la interfaz de selección del dataset, donde tenemos lo siguiente:

- Periodo temporal de interés (años y resolución horaria)
- Países incluidos en el análisis
- Tecnologías deseadas como: pv_national (solar fotovoltaica nacional), wind_national (eólica nacional), wind_onshore (eólica en tierra), wind_offshore (eólica marina).

El archivo se encuentra en CSV y presenta una estructura en la que tenemos las siguientes columnas:

- Tiempo (time), con marcas horarias
- Las anteriores especificaciones de distintos tipos de energía, pero con las iniciales del país correspondiente por combinaciones y seguido por el tipo de tecnología

Es decir, cada columna después de la primera columna (time o tiempo) comienza con las iniciales de cada país (por ejemplo: BA, BE, DE, entre otras) seguidas del tipo de tecnología y atributo. En total, el archivo incluye los datos para 36 países europeos y distintas tecnologías renovables. Una vez descargado el archivo CSV, este se almacenó en una carpeta accesible desde el entorno de PySpark.

Posteriormente, haremos una lectura que se realiza directamente desde una ubicación mediante instrucciones como: especificación de la ruta del archivo CSV y el uso del “spark.read ...”. A partir de esta lectura inicial, obtendremos un DataFrame en PySpark donde conservaremos la estructura original del archivo, sobre este DataFrame aplicaremos las etapas posteriores de normalización del esquema.

Estructura de las “tablas” de la BD

Tabla de hechos: Es el DataFrame principal, en formato “largo”, con los datos horarios: time (fecha y hora del registro), country (código del país), technology (tipo de tecnología), scenario (escenario del modelo), capacity_factor (factor de capacidad horario (0-1)). Cada fila representa un país, una tecnología y una hora correcta.

A partir de la tabla anterior, se usan columnas que funcionan como “dimensiones”: país (para agrupar y comparar países), tecnología (para separar solar y eólica), escenario (para comparar presente vs futuro) y tiempo (año, mes, día, hora). Todas estas “dimensiones” se manejan como columnas dentro de los DataFrames de PySpark, sin crear tablas aparte, pero sirven para estructurar las consultas y las agrupaciones del proyecto.

Código

```
from pyspark.sql import SparkSession

from pyspark.sql import functions as F

from pyspark.sql.functions import (
    col, lit, when, avg, first
)

from pyspark.sql.types import DoubleType

import pandas as pd

import matplotlib.pyplot as plt
```

```
import os
```

```
# 1. INICIALIZACIÓN DE SPARK
```

```
spark = (  
    SparkSession.builder  
        .appName("ClasificacionEnergiaEuropea_ProyectoPySpark")  
        .config("spark.executor.memory", "8g")  
        .getOrCreate()  
)
```

```
# 2. LECTURA DEL DATASET DESDE ARCHIVO LOCAL
```

```
base_path = os.path.dirname(os.path.abspath(__file__))  
ruta_local = os.path.join(base_path, "data", "Temperaturas.csv")
```

```
print(f"\nLeyendo archivo desde: {ruta_local}")
```

```
df_ancho_original = (  
    spark.read  
        .option("delimiter", ",")  
        .option("header", True)  
        .option("inferSchema", True)  
        .csv(f"file://{ruta_local}")
```

```
)
```

```
print("\n--- Vista Preliminar del DataFrame Original ---")  
df_ancho_original.show(5, truncate=False)  
print(f"Columnas totales: {len(df_ancho_original.columns)}")  
df_ancho_original.printSchema()
```

3. TRANSFORMACIÓN A FORMATO LARGO (PAÍS, TECNOLOGÍA)

```
TECNOLOGIAS_REQUERIDAS = ["pv_national", "wind_national"]
```

```
# Columnas de datos (todas menos 'time')
```

```
columnas_datos = [c for c in df_ancho_original.columns if c.lower() != "time"]
```

```
# Asumimos formato PAIS_TECNOLOGIA, ej: DE_pv_national
```

```
países_presentes = set(  
    c.split("_")[0]  
    for c in columnas_datos  
    if len(c.split("_")) > 1  
)
```

```
# Columnas que deberían existir
```

```
columnas_requeridas = set(  
    f"{país}_{tec}"  
    for país in países_presentes  
    for tec in TECNOLOGIAS_REQUERIDAS  
)
```

```

    for tec in TECNOLOGIAS_REQUERIDAS
)

# Columnas faltantes
columnas_faltantes = columnas_requeridas.difference(set(columnas_datos))

# Añadir columnas faltantes como NULL
df_ancho_unificado = df_ancho_original
for col_faltante in columnas_faltantes:
    df_ancho_unificado = df_ancho_unificado.withColumn(
        col_faltante,
        lit(None).cast(DoubleType())
    )

# Unpivot con stack: time, col_combinada, capacity_factor
columnas_para_stack = [c for c in df_ancho_unificado.columns if c.lower() !=
"time"]
num_columnas = len(columnas_para_stack)

stack_pairs = ", ".join([f'"{c}', `{c}`" for c in columnas_para_stack])
stack_expression = f"stack({num_columnas}, {stack_pairs})"

df_largo_analitico = (
    df_ancho_unificado
    .selectExpr(
        "time",
        f"stack({stack_expression}) as (col_combinada, capacity_factor)"
    )

```

```

    )
    .filter(col("capacity_factor").isNotNull())
)

# Separar country (2 letras) y technology (resto)
df_largo_analitico = (
    df_largo_analitico
    .withColumn("country", F.substring(col("col_combinada"), 1, 2))
    .withColumn(
        "technology",
        F.substring(col("col_combinada"), 4, F.length(col("col_combinada"))))
    )
    .select(
        "time",
        "country",
        "technology",
        col("capacity_factor").cast(DoubleType()).alias("capacity_factor")
    )
)

```

```

print("\n--- Vista Preliminar del DataFrame en Formato Largo (Analítico) ---")
df_largo_analitico.show(5, truncate=False)

```

```

# Extra: columna timestamp y mes (para procesamiento posteriores)
df_largo_con_fecha = (
    df_largo_analitico

```

```
.withColumn("timestamp", F.to_timestamp("time"))  
.withColumn("year", F.year("timestamp"))  
.withColumn("month", F.month("timestamp"))  
)
```

4. PROCESAMIENTO 1

Promedios anuales Solar vs Eólico + Clasificación por país

UMBRAL_MIXTURA = 0.10 # 10%

a) Agregado por país y tecnología (promedios)

```
df_agregado_tec = df_largo_analitico.groupBy("country", "technology").agg(  
    avg("capacity_factor").alias("avg_capacity_factor")  
)
```

```
print("\nPromedios por país y tecnología")
```

```
df_agregado_tec.show(10, truncate=False)
```

b) Pivot: una fila por país, columnas por tecnología

```
df_agregado_pais = (  
    df_agregado_tec  
    .groupBy("country")  
    .pivot("technology")  
    .agg(first("avg_capacity_factor"))  
)
```

```
print("\n--- Después del pivot (df_agregado_pais) ---")
```

```
df_agregado_pais.printSchema()
```

```
df_agregado_pais.show(10, truncate=False)
```

```
# c) Detección automática de columnas solar / eólica
```

```
cols_agregado = df_agregado_pais.columns
```

```
print("\nColumnas en df_agregado_pais:", cols_agregado)
```

```
solar_candidates = [c for c in cols_agregado if "pv" in c.lower()]
```

```
wind_candidates = [c for c in cols_agregado if "wind" in c.lower()]
```

```
if not solar_candidates or not wind_candidates:
```

```
    raise Exception(
```

```
        f"No pude encontrar columnas de pv/wind después del pivot. "
```

```
        f"Columnas encontradas: {cols_agregado}"
```

```
    )
```

```
solar_col = solar_candidates[0]
```

```
wind_col = wind_candidates[0]
```

```
print(f"\nUsando columna solar: {solar_col}")
```

```
print(f"Usando columna eólica: {wind_col}")
```

```
# d) Base para clasificación
```

```
df_base_clasif = df_agregado_pais.select(
```

```
"country",  
col(solar_col).alias("Solar_Avg"),  
col(wind_col).alias("Wind_Avg")  
)
```

e) Clasificación

```
df_clasificacion_final = (  
  df_base_clasif  
  .withColumn(  
    "Clasificacion",  
    when(  
      (col("Solar_Avg").isNotNull()) &  
      (col("Wind_Avg").isNotNull()) &  
      (col("Solar_Avg") > col("Wind_Avg")) &  
      ((col("Solar_Avg") - col("Wind_Avg")) / col("Solar_Avg") >  
lit(UMBRAL_MIXTURA)),  
      "Predominio Solar"  
    )  
    .when(  
      (col("Solar_Avg").isNotNull()) &  
      (col("Wind_Avg").isNotNull()) &  
      (col("Wind_Avg") > col("Solar_Avg")) &  
      ((col("Wind_Avg") - col("Solar_Avg")) / col("Wind_Avg") >  
lit(UMBRAL_MIXTURA)),  
      "Predominio Eólico"  
    )  
    .otherwise("Mixto o Balanceado")  
  )  
)
```



```
)  
.select("country", "Solar_Avg", "Wind_Avg", "Clasificacion")  
.sort("country")  
)
```

```
print("\nClasificación Final por País")  
df_clasificacion_final.show(50, truncate=False)
```

```
# ----- Gráficas P1 -----
```

```
df_pandas_clasificacion = df_clasificacion_final.toPandas()
```

```
# Figura 1: Comparación de promedios Solar vs Eólico
```

```
plt.figure(figsize=(18, 7))  
paises = df_pandas_clasificacion["country"]  
solar = df_pandas_clasificacion["Solar_Avg"]  
wind = df_pandas_clasificacion["Wind_Avg"]  
bar_width = 0.35  
index = range(len(paises))
```

```
plt.bar(  
    [i - bar_width / 2 for i in index],  
    solar,  
    bar_width,  
    label="Factor Cap. Solar Promedio",  
    color="#FFC300"
```

```

)
plt.bar(
    [i + bar_width / 2 for i in index],
    wind,
    bar_width,
    label="Factor Cap. Eólico Promedio",
    color="#581845"
)
plt.xlabel("País")
plt.ylabel("Factor de Capacidad Promedio (0-1)")
plt.title("Figura 1: Promedios Anuales Solar vs Eólico")
plt.xticks(list(index), paises, rotation=60, ha="right")
plt.legend()
plt.grid(axis="y", linestyle="--")
plt.tight_layout()
plt.savefig("P1_promedios_solar_vs_eolico.png")

```

Figura 2: Conteo de países por clasificación

```

plt.figure(figsize=(9, 6))
conteo_clasificacion = df_pandas_clasificacion["Clasificacion"].value_counts()
conteo_clasificacion.plot(kind="bar", color=["#28B463", "#3498DB", "#F39C12"])
plt.title("Figura 2: Conteo de Países por Predominio Energético")
plt.xlabel("Clasificación")
plt.ylabel("Número de Países")
plt.xticks(rotation=0)
plt.tight_layout()

```

```
plt.savefig("P1_conteo_clasificacion.png")
```

5. PROCESAMIENTO 2

Máximos y mínimos de factor de capacidad por país y tecnología

```
df_maxmin = df_largo_analitico.groupby("country", "technology").agg(  
    F.max("capacity_factor").alias("max_capacity_factor"),  
    F.min("capacity_factor").alias("min_capacity_factor")  
)
```

```
print("\nMáximos y mínimos por país y tecnología")
```

```
df_maxmin.show(20, truncate=False)
```

Pivot para tener columnas por tecnología (solo máximos para la gráfica)

```
df_maxmin_pivot = (  
    df_maxmin  
    .groupBy("country")  
    .pivot("technology")  
    .agg(first("max_capacity_factor"))  
)
```

```
cols_max = df_maxmin_pivot.columns
```

```
solar_max_candidates = [c for c in cols_max if "pv" in c.lower()]
```

```
wind_max_candidates = [c for c in cols_max if "wind" in c.lower()]
```

```
if solar_max_candidates and wind_max_candidates:
```

```
    solar_max_col = solar_max_candidates[0]
```

```
    wind_max_col = wind_max_candidates[0]
```

```
df_maxmin_graph = df_maxmin_pivot.select(
```

```
    "country",
```

```
    col(solar_max_col).alias("Solar_Max"),
```

```
    col(wind_max_col).alias("Wind_Max")
```

```
).sort("country")
```

```
df_pandas_max = df_maxmin_graph.toPandas()
```

```
plt.figure(figsize=(18, 7))
```

```
países = df_pandas_max["country"]
```

```
solar_max = df_pandas_max["Solar_Max"]
```

```
wind_max = df_pandas_max["Wind_Max"]
```

```
bar_width = 0.35
```

```
index = range(len(países))
```

```
plt.bar(
```

```
    [i - bar_width / 2 for i in index],
```

```
    solar_max,
```

```
    bar_width,
```

```
    label="Máx. Solar",
```

```
    color="#F1C40F"
```

```

)
plt.bar(
    [i + bar_width / 2 for i in index],
    wind_max,
    bar_width,
    label="Máx. Eólico",
    color="#2E86C1"
)
plt.xlabel("País")
plt.ylabel("Máx. Factor de Capacidad (0-1)")
plt.title("Figura 3: Máximos Anuales de Factor de Capacidad por País")
plt.xticks(list(index), paises, rotation=60, ha="right")
plt.legend()
plt.grid(axis="y", linestyle="--")
plt.tight_layout()
plt.savefig("P2_maximos_solar_vs_eolico.png")

```

6. PROCESAMIENTO 3

Desviación estándar (volatilidad) por país y tecnología

```

df_std = df_largo_analitico.groupby("country", "technology").agg(
    F.stddev("capacity_factor").alias("std_capacity_factor")
)

```

```
print("\nDesviación estándar por país y tecnología")
```

```
df_std.show(20, truncate=False)
```

```
df_std_pivot = (  
    df_std  
    .groupBy("country")  
    .pivot("technology")  
    .agg(first("std_capacity_factor"))  
)
```

```
cols_std = df_std_pivot.columns
```

```
solar_std_candidates = [c for c in cols_std if "pv" in c.lower()]
```

```
wind_std_candidates = [c for c in cols_std if "wind" in c.lower()]
```

```
if solar_std_candidates and wind_std_candidates:
```

```
    solar_std_col = solar_std_candidates[0]
```

```
    wind_std_col = wind_std_candidates[0]
```

```
df_std_graph = df_std_pivot.select(  
    "country",  
    col(solar_std_col).alias("Solar_STD"),  
    col(wind_std_col).alias("Wind_STD")  
)  
.sort("country")
```

```
df_pandas_std = df_std_graph.toPandas()
```

```

plt.figure(figsize=(18, 7))

países = df_pandas_std["country"]
solar_std = df_pandas_std["Solar_STD"]
wind_std = df_pandas_std["Wind_STD"]
bar_width = 0.35
index = range(len(países))

plt.bar(
    [i - bar_width / 2 for i in index],
    solar_std,
    bar_width,
    label="STD Solar",
    color="#8E44AD"
)

plt.bar(
    [i + bar_width / 2 for i in index],
    wind_std,
    bar_width,
    label="STD Eólico",
    color="#16A085"
)

plt.xlabel("País")
plt.ylabel("Desviación Estándar del Factor de Capacidad")
plt.title("Figura 4: Volatilidad de Solar vs Eólico por País")
plt.xticks(list(index), países, rotation=60, ha="right")
plt.legend()

```

```
plt.grid(axis="y", linestyle="--")
plt.tight_layout()
plt.savefig("P3_std_solar_vs_eolico.png")
```

7. PROCESAMIENTO 4

Promedios mensuales por tecnología (perfil estacional)

```
df_mensual = (
    df_largo_con_fecha
    .groupBy("month", "technology")
    .agg(
        avg("capacity_factor").alias("avg_capacity_factor")
    )
)

print("\nPromedios mensuales por tecnología")
df_mensual.orderBy("month", "technology").show(24, truncate=False)

df_mensual_pivot = (
    df_mensual
    .groupBy("month")
    .pivot("technology")
    .agg(first("avg_capacity_factor"))
    .orderBy("month")
)
```



```
cols_mens = df_mensual_pivot.columns
solar_mens_candidates = [c for c in cols_mens if "pv" in c.lower()]
wind_mens_candidates = [c for c in cols_mens if "wind" in c.lower()]
```

```
if solar_mens_candidates and wind_mens_candidates:
```

```
    solar_mens_col = solar_mens_candidates[0]
```

```
    wind_mens_col = wind_mens_candidates[0]
```

```
df_mens_graph = df_mensual_pivot.select(
    "month",
    col(solar_mens_col).alias("Solar_Mensual"),
    col(wind_mens_col).alias("Wind_Mensual")
).orderBy("month")
```

```
df_pandas_mens = df_mens_graph.toPandas()
```

```
plt.figure(figsize=(10, 6))
```

```
meses = df_pandas_mens["month"]
```

```
solar_mens = df_pandas_mens["Solar_Mensual"]
```

```
wind_mens = df_pandas_mens["Wind_Mensual"]
```

```
plt.plot(meses, solar_mens, marker="o", label="Solar", linestyle="-")
```

```
plt.plot(meses, wind_mens, marker="s", label="Eólico", linestyle="--")
```

```
plt.xlabel("Mes")
```

```
plt.ylabel("Factor de Capacidad Promedio (0-1)")
```

```
plt.title("Figura 5: Perfil Mensual Solar vs Eólico")
plt.xticks(list(meses))
plt.grid(True, linestyle="--", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.savefig("P4_promedios_mensuales_solar_vs_eolico.png")
```

8. PROCESAMIENTO 5

```
# Ranking de países según "score medio" (promedio Solar+Eólico)
```

```
df_ranking = (
    df_clasificacion_final
    .withColumn(
        "Score_Medio",
        (col("Solar_Avg") + col("Wind_Avg")) / 2.0
    )
    .orderBy(col("Score_Medio").desc())
)
```

```
print("\nRanking de países por Score Medio")
```

```
df_ranking.show(50, truncate=False)
```

```
df_pandas_rank = df_ranking.toPandas()
```

```
# Tomamos top 10 para la gráfica
```

```

df_pandas_top10 = df_pandas_rank.head(10)

plt.figure(figsize=(10, 6))
plt.bar(
    df_pandas_top10["country"],
    df_pandas_top10["Score_Medio"],
    color="#E67E22"
)
plt.xlabel("País")
plt.ylabel("Score Medio (Promedio Solar + Eólico)")
plt.title("Figura 6: Top 10 Países por Score Energético Medio")
plt.grid(axis="y", linestyle="--", alpha=0.6)
plt.tight_layout()
plt.savefig("P5_ranking_top10_score_medio.png")

```

9. CIERRE

```

spark.stop()

print("\nProceso de PySpark completado y sesión de Spark detenida.")
print("Archivos de resultados guardados:")
print(" - P1_promedios_solar_vs_eolico.png")
print(" - P1_conteo_clasificacion.png")
print(" - P2_maximos_solar_vs_eolico.png")
print(" - P3_std_solar_vs_eolico.png")
print(" - P4_promedios_mensuales_solar_vs_eolico.png")
print(" - P5_ranking_top10_score_medio.png")

```

Resultados (Resultados de consultas y gráficas)

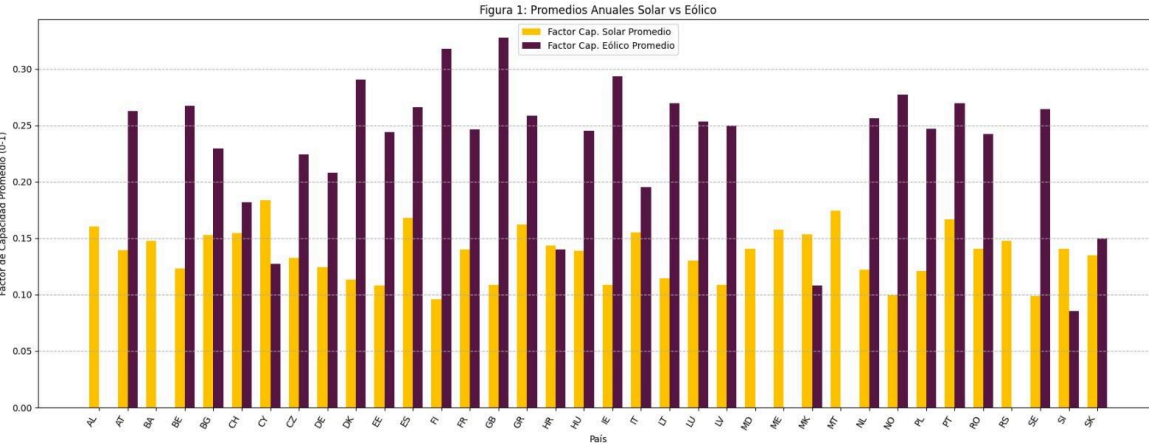
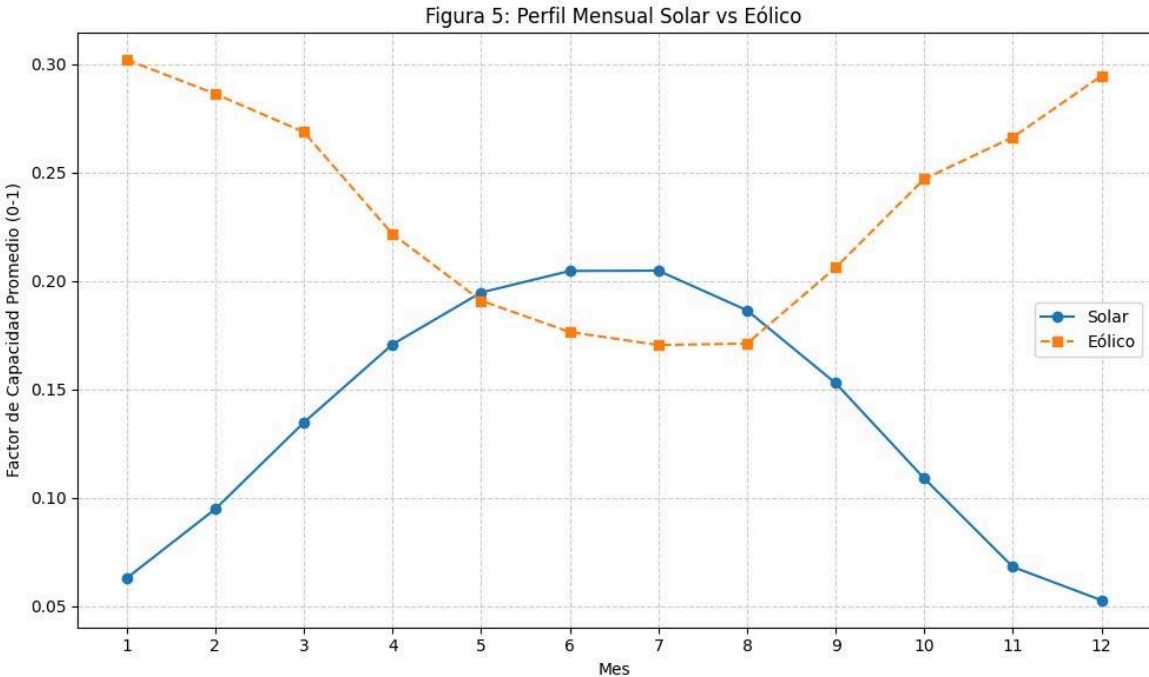


Figura 2: Conteo de Países por Predominio Energético

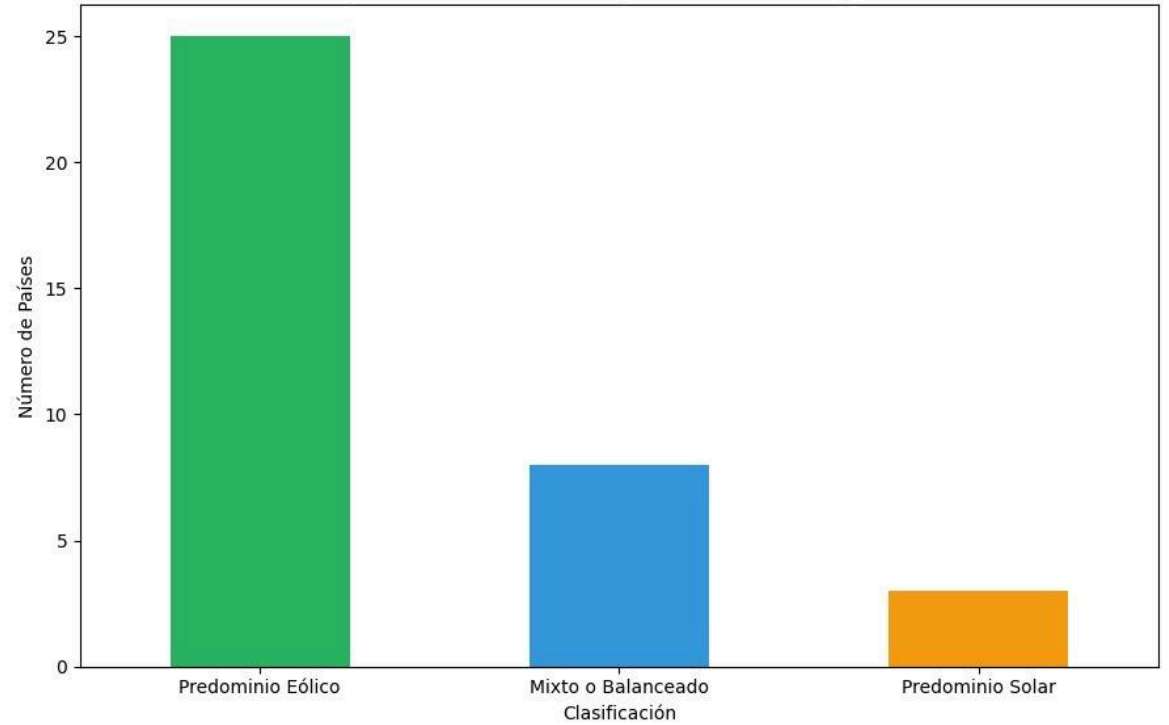
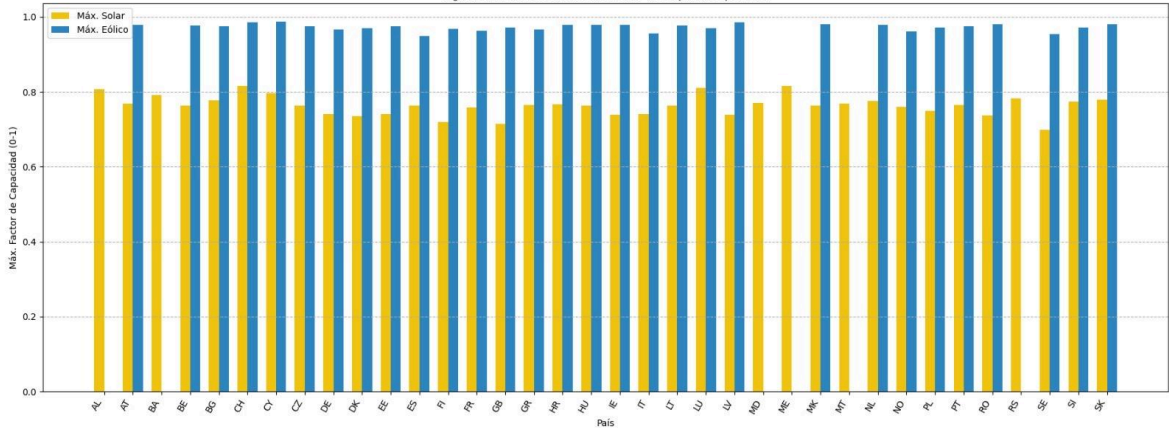
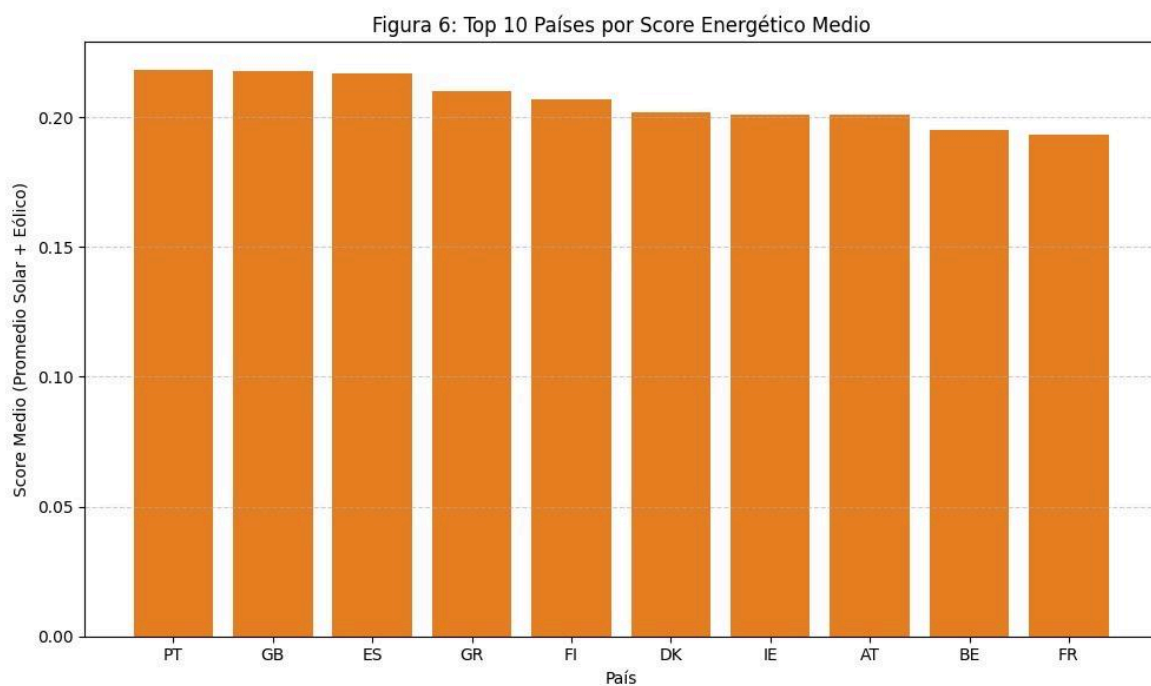
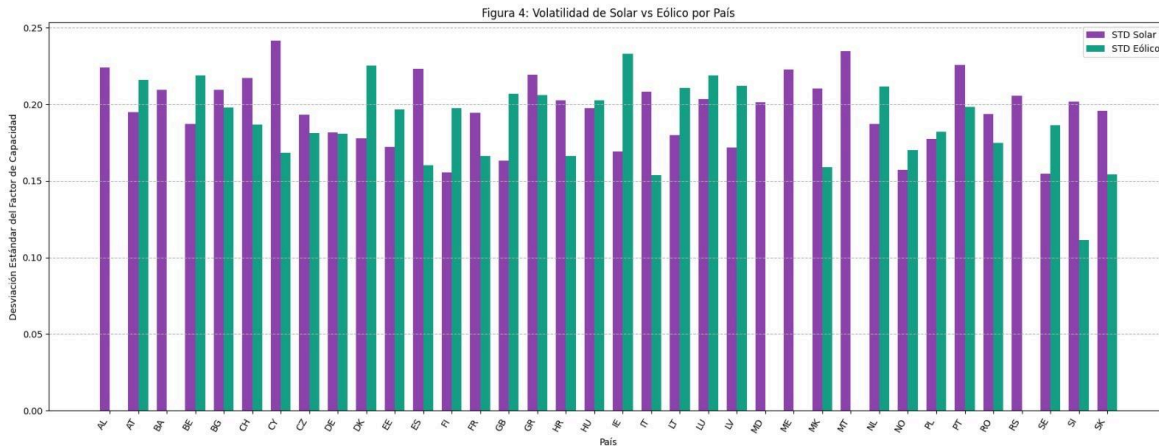


Figura 3: Máximos Anuales de Factor de Capacidad por País





Conclusiones

El proyecto concluyó exitosamente con la clasificación de países europeos según su predominio solar o eólico, demostrando de manera efectiva la capacidad de PySpark para el análisis de Big Data. Se superó el desafío de procesar un dataset masivo y heterogéneo mediante la unificación del esquema y la transformación a un formato analítico.

El enfoque en PySpark fue crucial, ya que permitió la ejecución distribuida de procesos de limpieza y el cálculo rápido de métricas agregadas como el factor de capacidad promedio. Estos resultados estadísticos, combinados con una

visualización clara mediante Matplotlib, no solo nos ayudaron a entender el uso de energías renovables en el continente Europeo, sino que también validaron un modelo

Bibliografía:

- Open Power System Data. Time series – European renewable energy profiles (Ninja PV & Wind Profiles). Proyecto de datos abiertos de sistemas eléctricos europeos.
- Pfenninger, S., & Staffell, I. (2016). Long-term patterns of European PV output using 30 years of validated hourly reanalysis and satellite data. *Energy*, 114, 1251–1265. (Base metodológica de Renewables.ninja).
- Apache Software Foundation. Apache Spark™ Documentation. Sección PySpark (Spark SQL, DataFrames and Datasets).
- The Pandas Development Team. Pandas Documentation. Uso de DataFrames en Python para análisis de datos.
- Hunter, J. D. et al. Matplotlib Documentation. Librería de gráficos en 2D para Python.
- McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (2ª ed.). O'Reilly Media.